Vision-Based Path Following with Multi-Controller Architecture

Shlok Mehndiratta

March 21, 2026

Repository: `ugv_uav_control`
Platform: ROS 2 Jazzy, Gazebo Sim (Harmonic), Python 3.12
Document Version: v1.1

Contents

1 Introduction **5**

1.1 Problem Context and Motivation 5

1.2 System Overview: UAV–UGV Architecture 5

1.3 Why This Problem Motivates Uncertainty-Aware Control 5

2 System Modeling and Setup **6**

2.1 Robot Kinematic Model 6

2.2 Perception Pipeline 6

2.3 Deterministic Assumptions (Early Approach) 7

3 Early Control Approaches: Geometric and Deterministic Methods **7**

3.1 Pure Pursuit 7

3.2 Stanley Controller 7

3.3 Deterministic Model Predictive Control (MPC) 8

3.4 Integration Method 8

4 Limitations of Deterministic Control **9**

4.1 Observability Loss Under Intermittent Sensing . . . 9
4.2 Unmodeled Noise Induces Overconfidence . . . 9
4.3 Actuator Lag Creates Prediction–Reality Gap . . . 10
4.4 The Need for Explicit Uncertainty Modeling . . . 10

5 Transition to Belief-Space Formulation 10
5.1 Why Model Predictive Control for Belief-Space Problems . . . 10
5.2 From State to Belief . . . 11
5.3 Partially Observable System Formulation . . . 11
5.4 Role of State Estimation . . . 12
5.5 Observability as an Action-Dependent Property . . . 12
5.6 Solution Strategy: Belief-Space MPC . . . 13

6 Belief-Space Model Predictive Control 13
6.1 Theoretical Foundation . . . 13
6.1.1 From Deterministic to Stochastic Control . . . 13
6.1.2 Key Assumptions . . . 14
6.2 State Estimation: Extended Kalman Filter . . . 14
6.3 Belief-Space Cost Function . . . 15
6.3.1 Uncertainty Cost Interpretation . . . 15
6.4 Covariance Propagation Within Horizon . . . 16
6.5 Action-Dependent Observability . . . 16
6.5.1 FoV Model . . . 16
6.5.2 Smooth Observability Function . . . 16
6.5.3 Covariance Blending . . . 16
6.6 Optimization Problem . . . 17
6.7 Actuator Dynamics . . . 17

7 Implementation Architecture 17
7.1 System Pipeline . . . 17
7.2 ROS 2 Node Structure . . . 18
7.3 Layered Architecture . . . 18

8 Experimental Evaluation **19**

8.1 Experimental Setup . . . 19

8.2 Results . . . 19

8.3 Result Interpretation . . . 19

8.3.1 Tracking Performance . . . 19

8.3.2 Control Smoothness . . . 20

8.3.3 Covariance Behavior . . . 20

8.3.4 FoV Maintenance . . . 20

8.4 Effect of Uncertainty Weight . . . 21

8.5 Belief-Space Control Loop . . . 21

8.6 Visualization of Results . . . 21

9 Discussion **22**

9.1 Why Belief-Space Control Outperforms Deterministic Approaches . . . 22

9.1.1 Jacobian Coupling Drives Smooth Control . . . 22

9.1.2 Emergent FoV Awareness . . . 23

9.1.3 Modest Tracking Improvement . . . 23

9.2 Physical Interpretation of Results . . . 24

9.2.1 Why Covariance Traces are Comparable . . . 24

9.2.2 Why Dropout Duration Improves . . . 24

9.3 Limitations and Scope of Results . . . 24

9.3.1 Simulation vs. Reality . . . 24

9.3.2 Steady-State Behavior . . . 25

9.3.3 Optimization Landscape . . . 25

9.4 What This Work Achieves . . . 25

9.5 What Remains Unsolved . . . 25

10 System Constraints and Limitations **26**

10.1 Kinematic and Actuation Constraints . . . 26

10.1.1 Non-Holonomic Motion . . . 26

10.1.2 Actuator Constraints . . . 26

10.2 Perception Limitations . . . 26

10.2.1 Binary Thresholding Fragility . . . 26
10.2.2 ArUco Detection Limits . . . 27
10.2.3 FoV Boundary Effects . . . 27
10.3 Estimation Limitations . . . 27
10.3.1 EKF Gaussianity Assumption . . . 27
10.3.2 Noise Model Tuning . . . 28
10.3.3 Linearization Error Over Horizon . . . 28
10.4 Control System Limitations . . . 28
10.4.1 Non-Convex Optimization . . . 28
10.4.2 Computational Latency . . . 28
10.4.3 No Terminal Cost . . . 28
10.5 System-Level Assumptions . . . 29
10.5.1 Static Aerial Platform . . . 29
10.5.2 Obstacle-Free Environment . . . 29
10.5.3 Sufficient Initial FoV Overlap . . . 29
10.5.4 Vertical Camera Assumption . . . 29
10.6 Summary of Limitations . . . 29

11 Future Research Directions **30**
11.1 Near-Term Engineering Improvements . . . 30
11.2 Theoretical Completions . . . 30
11.3 Estimation Extensions . . . 31
11.4 Coordinated UAV–UGV Planning . . . 31
11.5 Learning-Based Augmentations . . . 31
11.6 Robustness Extensions . . . 32
11.7 Sim-to-Real Transfer . . . 32

Appendices **33**

A Symbol Definitions **33**

A Sensor Specifications **34**

A Implementation Notes **34**

1 Introduction

1.1 Problem Context and Motivation

Autonomous path following represents a fundamental task in mobile robotics. The core challenge addressed by this work is enabling a differential-drive ground robot (TurtleBot3 Burger) to follow a predefined path using vision-based localization and control, without relying on GPS, SLAM, or external odometry correction.

The key constraints are:

- **Localization**: fiducial marker (ArUco) detection from an overhead camera, available only when the robot is within the field-of-view (FoV).
- **Perception**: path geometry extraction from camera imagery using image processing.
- **Control**: steering the robot to track path waypoints while respecting kinematic constraints.

This problem is relevant to industrial inspection, warehouse automation, and coordinated UAV–UGV systems where vision-based relative localization is preferred to absolute positioning.

1.2 System Overview: UAV–UGV Architecture

The system consists of two platforms:

- **Ground robot (UGV)**: TurtleBot3 Burger differential-drive platform, executing path-following control.
- **Aerial platform (UAV)**: stationary overhead camera providing top-down perspective for pose and path sensing.

This architecture imposes a fundamental observability challenge: the robot can only be localized when visible to the aerial camera. Once the robot exits the FoV, localization relies solely on wheel odometry, which drifts unbounded without correction.

1.3 Why This Problem Motivates Uncertainty-Aware Control

Early approaches treated the system deterministically, assuming full observability and perfect state knowledge. In practice, three factors expose this assumption as fragile:

1. **Perception is intermittent**: ArUco measurements are available only within the FoV. Outside this region, the robot has no direct external feedback.
2. **Sensor noise is non-negligible**: both the overhead camera (marker detection) and wheel odometry contribute measurement error. Ignoring this noise leads to overconfident control.

3. **Dynamics are non-ideal**: actuator latency, wheel slip, and sensor quantization introduce real uncertainty. A controller that ignores these effects cannot adapt robustly when uncertainty grows.

These observations motivate the transition from deterministic control to explicit uncertainty modeling, formalized through belief-space control. The remainder of this document traces this evolution and presents the resulting framework.

2 System Modeling and Setup

2.1 Robot Kinematic Model

The TurtleBot3 Burger is modeled as a planar unicycle:

$$\mathbf{x} = \begin{bmatrix} x & y & \theta \end{bmatrix}^T \in \mathbb{R}^2 \times S^1,$$

where (x, y) is position in the world frame and θ is heading. Kinematics are:

$$\dot{x} = v\cos\theta, \quad \dot{y} = v\sin\theta, \quad \dot{\theta} = \omega.$$

The non-holonomic constraint (no lateral motion) is implicit in this formulation. Control inputs are (v, ω) with bounds:

$$0 \leq v \leq 0.22 \text{ m/s}, \quad |\omega| \leq 2.0 \text{ rad/s}.$$

Physical parameters: wheel separation $L = 0.160$ m, wheel radius $r = 0.033$ m, mass $m = 0.825$ kg.

2.2 Perception Pipeline

Path localization uses an overhead camera mounted at height $h = 10$ m with 60 horizontal FoV. The pipeline consists of:

1. **ArUco fiducial detection**: Robot pose estimated from a 0.5×0.5 m ArUco marker mounted 0.2 m above the base.
2. **Path segmentation**: Binary thresholding (threshold 100) followed by floodfill to extract lane region.
3. **Centerline extraction**: Distance transform finds the path centerline as the locus of maximum distance to boundaries.
4. **Waypoint generation**: Waypoints computed in robot body frame at target distance and bearing.

The camera optical axis is assumed perfectly vertical (no tilt correction). Heading is estimated from the marker orientation; accuracy degrades at large camera offsets due to perspective distortion.

2.3 Deterministic Assumptions (Early Approach)

Initial controller development assumed:

- State is fully known (no estimation error).
- Camera provides continuous pose measurements.
- Sensor noise is negligible.
- Actuators respond instantly to commands.

Under these assumptions, control reduces to a tracking problem: steer to minimize cross-track and heading error relative to path geometry. Three controllers were explored under this framework.

3 Early Control Approaches: Geometric and Deterministic Methods

Under the deterministic assumption, three path-following controllers were evaluated. Each represents an increasing level of sophistication.

3.1 Pure Pursuit

Pure Pursuit is a geometric controller that computes a circular arc to a look-ahead waypoint. The steering angle is:

$$\gamma = \frac{2y}{L_d^2}, \quad L_d = \sqrt{x^2 + y^2},$$

where y is lateral offset and L_d is distance to waypoint, both in robot frame. The commanded angular velocity is:

$$\omega = v_{\text{target}} \cdot \gamma.$$

Advantages: Simple to implement, noise-robust (geometric rather than analytic).

Limitation: Does not explicitly correct heading error; robot may approach path misaligned.

3.2 Stanley Controller

The Stanley controller (from Stanford DARPA Grand Challenge) minimizes both cross-track error (CTE) and heading error:

$$\delta = \psi_e + \arctan\left(\frac{k_{\text{cte}} \cdot e_{\text{cte}}}{1 + v}\right),$$

where ψ_e is heading error, e_{cte} is lateral error, and $k_{\text{cte}} = 2.0$. The steering command is:

$$\omega = \frac{v \tan(\delta)}{0.2}.$$

Advantages: Convergence guaranteed for bounded initial error; velocity-dependent damping improves stability.

Limitation: Assumes knowledge of path tangent; in practice, tangent estimated from waypoint orientation.

3.3 Deterministic Model Predictive Control (MPC)

MPC extends the framework to handle actuation constraints and multi-step planning. **From this point onward, the system state is augmented to include actuator dynamics, expanding the state dimension from 3D to 5D** ($\mathbf{x} \in \mathbb{R}^5$):

$$\mathbf{x}_{\text{aug}} = [x, y, \theta, v_{\text{act}}, \omega_{\text{act}}]^T.$$

First-order actuator dynamics are modeled:

$$\dot{v}_{\text{act}} = \frac{v_{\text{cmd}} - v_{\text{act}}}{\tau_v}, \quad \dot{\omega}_{\text{act}} = \frac{\omega_{\text{cmd}} - \omega_{\text{act}}}{\tau_\omega},$$

with time constants $\tau_v = 0.5$ s and $\tau_\omega = 0.2$ s.

The MPC cost function is:

$$J = \sum_{k=0}^{N-1} \left[w_d \|\mathbf{p}_k - \mathbf{p}_{\text{target}}\|^2 + w_h (\theta_k - \theta_{\text{target}})^2 + w_r \|\Delta \mathbf{u}_k\|^2 \right],$$

with weights $w_d = 10$, $w_h = 2$, $w_r = 5$. Horizon: $N = 5$ steps, $dt = 0.2$ s. Optimization via SLSQP.

Advantages: Explicit constraint handling; multi-step lookahead; smooth control from rate penalties.

Limitations: Non-convex optimization; local optima only. No explicit handling of uncertainty or observability.

3.4 Integration Method

For accurate trajectory prediction, the unicycle kinematics are integrated exactly for nonzero angular velocity:

$$x_{k+1} = x_k + \frac{v}{\omega}[\sin(\theta_k + \omega\, dt) - \sin\theta_k],$$
$$y_{k+1} = y_k - \frac{v}{\omega}[\cos(\theta_k + \omega\, dt) - \cos\theta_k],$$

with straight-line approximation for $|\omega| < 10^{-4}$ to avoid division by zero.

Forward Euler integration is replaced by exact arc integration to eliminate $O(dt)$ discretization error on curved trajectories.

4 Limitations of Deterministic Control

The deterministic approaches work well in controlled settings with continuous external sensing. In practice, three critical limitations emerge:

4.1 Observability Loss Under Intermittent Sensing

The robot is localized via ArUco detection, which is available only when the robot is within the UAV's FoV. The FoV has a circular boundary with radius:

$$r_{\text{fov}} = h \tan\left(\frac{\phi}{2}\right) \approx 5.77 \text{ m}$$

for $h = 10$ m and $\phi = 60$.

Deterministic MPC plans trajectories without explicitly considering this observability boundary. The robot can approach or exit the FoV, at which point localization fails. Once outside:

- External (ArUco) pose feedback is lost.
- Control must rely on wheel odometry alone.
- Odometry drifts unbounded without correction.
- Uncertainty in state grows without bound.

Empirical observation: Deterministic MPC occasionally approaches FoV boundaries and experiences transient localization loss, degrading control performance.

4.2 Unmodeled Noise Induces Overconfidence

Both sensors introduce real measurement noise:

- ArUco marker detection has finite precision (marker orientation estimation error ≈ 0.01 rad).
- Wheel odometry integrates motor control errors and wheel slip (cumulative drift $\approx 0.5\%$ per meter).

Deterministic control ignores this noise. The controller issues commands assuming perfect state knowledge, leading to:

- Aggressive maneuvers that assume noise-free sensing.

- Lack of robustness when noise levels exceed assumed tolerances.
- No natural mechanism to trade off control aggressiveness for state confidence.

4.3 Actuator Lag Creates Prediction–Reality Gap

Real actuators do not respond instantly. The model includes first-order dynamics ($\tau_v = 0.5$ s, $\tau_\omega = 0.2$ s), yet MPC still assumes these dynamics are perfectly understood and modeled. In reality:

- Motor response varies with battery voltage and load.
- Wheel slip depends on ground conditions and varies between wheels.
- Unmodeled friction and damping affect dynamic response.

Deterministic MPC does not account for the uncertainty *about* these dynamics. It assumes the model is true, not approximately true.

4.4 The Need for Explicit Uncertainty Modeling

These three failures reveal a common root cause: the controller must know how much it can trust the state estimate. This requires:

1. **Quantification of uncertainty**: represent state confidence as a distribution (belief), not a point estimate.
2. **Propagation of uncertainty**: compute how uncertainty evolves over the MPC horizon under control actions.
3. **Observability-aware planning**: choose actions that maintain observability (stay in FoV) or actively improve state confidence.
4. **Uncertainty-weighted control**: penalize maneuvers that amplify uncertainty, inducing conservative behavior when confidence is low.

This is the problem formulation of belief-space control.

5 Transition to Belief-Space Formulation

5.1 Why Model Predictive Control for Belief-Space Problems

The problem involves three key features that motivate MPC: (i) nonlinear dynamics (unicycle kinematics), (ii) hard constraints on actuation and FoV, and (iii) future-dependent observability (remaining in FoV is a control decision). Reactive or linear controllers cannot simultaneously

optimize long-term observability and tracking. Belief-space MPC addresses this by planning over future belief trajectories, explicitly trading off immediate tracking performance for robust long-term observability.

5.2 From State to Belief

The fundamental shift is from point-estimate state control to probabilistic belief control. Rather than assuming a known state $\mathbf{x}_k$, we model uncertainty explicitly.

Belief Definition A belief is a probability distribution over the state space:

$$\mathbf{b}_k = (\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k),$$

where $\boldsymbol{\mu}_k$ is the mean (expected state) and $\boldsymbol{\Sigma}_k$ is the covariance (state uncertainty).

For Gaussian beliefs:

$$\mathbf{b}_k \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

The belief state is (5×1) for mean and (5×5) for covariance, accounting for position, heading, and actuator states.

5.3 Partially Observable System Formulation

The system evolves under stochastic dynamics with partial observations:

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k) + \mathbf{w}_k, \quad \mathbf{w}_k \sim \mathcal{N}(0, \mathbf{Q}) \tag{1}$$

$$\mathbf{z}_k = h(\mathbf{x}_k) + \mathbf{v}_k, \quad \mathbf{v}_k \sim \mathcal{N}(0, \mathbf{R}) \tag{2}$$

where:

- $\mathbf{w}_k$ is process noise (unmodeled dynamics).
- $\mathbf{z}_k$ is a measurement, available only when the robot is in FoV.
- $\mathbf{v}_k$ is measurement noise (sensor imprecision).

The control objective becomes:

$$\min_{\mathbf{u}_{0:N-1}} \mathbb{E}\left[\sum_{k=0}^{N-1} c(\mathbf{x}_k, \mathbf{u}_k)\right],$$

optimization over control sequence given only observations $\mathbf{z}_0, \ldots, \mathbf{z}_{k-1}$, not true state.

5.4 Role of State Estimation

Since the true state is unobservable, we maintain an estimate $\hat{\mathbf{b}}_k$ of the belief. This is updated recursively:

1. **Prediction**: propagate uncertainty forward under dynamics and process noise

$$\mathbf{\Sigma}_{k+1|k} = \mathbf{F}_k \mathbf{\Sigma}_{k|k} \mathbf{F}_k^T + \mathbf{Q},$$

   where $\mathbf{F}_k = \frac{\partial f}{\partial \mathbf{x}}\Big|_{\boldsymbol{\mu}_{k|k}}$ is the Jacobian.

2. **Update**: when a measurement arrives, reduce uncertainty

$$\mathbf{\Sigma}_{k|k} = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{\Sigma}_{k|k-1},$$

   where $\mathbf{K}$ is the Kalman gain.

The Extended Kalman Filter (EKF) implements this recursion by linearizing f and h around the current mean estimate.

5.5 Observability as an Action-Dependent Property

The key insight is that *observability depends on the robot's trajectory*, which is controlled.

When the robot is inside the FoV:

- ArUco measurements are available.
- Kalman update reduces covariance.
- Uncertainty can be controlled (information-gathering).

When the robot is outside the FoV:

- Only odometry is available (open-loop).
- Kalman prediction grows covariance.
- Uncertainty grows unbounded without external sensing.

Action-dependent observability: The control input $\mathbf{u}_k$ determines the robot's future position, which determines future observability. MPC can choose trajectories that *stay in FoV*, maintaining observability implicitly through the cost function rather than explicit constraints.

5.6 Solution Strategy: Belief-Space MPC

Belief-space MPC addresses this challenge by:

1. **Running EKF in parallel**: maintain current belief $\mathbf{b}_k$ updated from measurements.
2. **Predicting belief over horizon**: for each candidate control sequence, simulate belief propagation.
3. **Costing uncertainty growth**: add a penalty term $\lambda \operatorname{tr}(\boldsymbol{\Sigma}_k)$ that penalizes large covariances.
4. **Optimization over controls**: choose $\mathbf{u}_{0:N-1}$ that minimizes tracking error + uncertainty cost.

The result is a controller that automatically trades off tracking accuracy for state confidence, maintains observability implicitly, and adapts aggressiveness based on how much it trusts its state estimate.

6 Belief-Space Model Predictive Control

6.1 Theoretical Foundation

6.1.1 From Deterministic to Stochastic Control

The system is partially observable with stochastic dynamics:

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k, \mathbf{w}_k) \quad \text{(system evolution)} \tag{3}$$

$$\mathbf{z}_k = h(\mathbf{x}_k, \mathbf{v}_k) \quad \text{(measurement)} \tag{4}$$

where $\mathbf{w}_k \sim \mathcal{N}(0, \mathbf{Q})$ and $\mathbf{v}_k \sim \mathcal{N}(0, \mathbf{R})$ are process and measurement noise.

The control objective is:

$$\min_{\mathbf{u}_{0:N-1}} \mathbb{E}\left[\sum_{k=0}^{N-1} c(\mathbf{x}_k, \mathbf{u}_k)\right].$$

Since the true state is unobservable, optimization occurs over the belief $\mathbf{b}_k = (\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$ using:

1. **Gaussian approximation**: EKF linearizes f and h around current mean
2. **Receding horizon**: MPC optimizes over N steps with re-planning at each step

This approach is valid under small noise and localized operations.

6.1.2 Key Assumptions

Four assumptions enable this framework:

A1: Gaussian belief: State distribution $\mathbf{b}_k \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$. *Justification*: natural for linearized filtering. *Limitation*: fails for multi-modal or heavy-tailed distributions.

A2: Small noise: Linearization error remains bounded over horizon. *Justification*: noise covariances empirically small. *Limitation*: invalid under large disturbances.

A3: Unimodal belief: no filter divergence over horizon. *Justification*: EKF re-initializes on large residuals. *Limitation*: does not handle kidnapped robot.

A4: Known noise models: $\mathbf{Q}$ and $\mathbf{R}$ precisely tuned. *Justification*: empirically calibrated. *Limitation*: no online adaptation.

6.2 State Estimation: Extended Kalman Filter

The EKF approximates nonlinear belief propagation by linearizing dynamics around the mean:

$$f(\mathbf{x}) \approx f(\boldsymbol{\mu}) + \mathbf{F}(\mathbf{x} - \boldsymbol{\mu}), \quad \mathbf{F} = \left.\frac{\partial f}{\partial \mathbf{x}}\right|_{\boldsymbol{\mu}}.$$

For the unicycle with actuator states $\mathbf{x}_k = [x, y, \theta, v_{\text{act}}, \omega_{\text{act}}]^T$, the state transition Jacobian is:

$$\mathbf{F} = \begin{bmatrix} 1 & 0 & -v\sin\theta\, dt & \cos\theta\, dt & 0 \\ 0 & 1 & v\cos\theta\, dt & \sin\theta\, dt & 0 \\ 0 & 0 & 1 & 0 & dt \\ 0 & 0 & 0 & 1-\alpha_v dt & 0 \\ 0 & 0 & 0 & 0 & 1-\alpha_\omega dt \end{bmatrix}.$$

The coupling terms $\mathbf{F}[0,2]$ and $\mathbf{F}[1,2]$ capture the physical effect: heading uncertainty couples into position uncertainty through the kinematics.

The EKF update cycle is:

Prediction

$$\boldsymbol{\mu}_{k+1|k} = f(\boldsymbol{\mu}_{k|k}, \mathbf{u}_k) \tag{5}$$

$$\boldsymbol{\Sigma}_{k+1|k} = \mathbf{F}_k \boldsymbol{\Sigma}_{k|k} \mathbf{F}_k^\top + \mathbf{Q} \tag{6}$$

Update (when measurement available)

$$\mathbf{K} = \mathbf{\Sigma}_{k|k-1}\mathbf{H}^\top(\mathbf{H}\mathbf{\Sigma}_{k|k-1}\mathbf{H}^\top + \mathbf{R})^{-1} \tag{7}$$

$$\boldsymbol{\mu}_{k|k} = \boldsymbol{\mu}_{k|k-1} + \mathbf{K}(\mathbf{z} - \mathbf{H}\boldsymbol{\mu}_{k|k-1}) \tag{8}$$

$$\mathbf{\Sigma}_{k|k} = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{\Sigma}_{k|k-1} \tag{9}$$

Noise models (empirically tuned):

$$\mathbf{Q} = \mathrm{diag}(10^{-4}, 10^{-4}, 4\times10^{-4}, 10^{-4}, 2.5\times10^{-3}),$$

$$\mathbf{R}_{\text{aruco}} = \mathrm{diag}(0.0025, 0.0025, 0.01, 0.0004, 0.0025), \quad \mathbf{R}_{\text{odom}} = \mathrm{diag}(0.0004, 0.0025).$$

The EKF is valid locally when linearization error is small relative to noise. Over long horizons, error accumulates; MPC replanning helps mitigate this.

6.3 Belief-Space Cost Function

Deterministic MPC minimized:

$$J_{\text{det}} = \sum_{k=0}^{N-1} \left[w_d \|\boldsymbol{\mu}_k - \mathbf{p}_{\text{target}}\|^2 + w_h(\theta_k - \theta_{\text{target}})^2 + w_r\|\Delta\mathbf{u}_k\|^2 \right].$$

Belief-space MPC adds uncertainty penalty:

$$J_{\text{belief}} = \sum_{k=0}^{N-1} \left[w_d \|\boldsymbol{\mu}_k - \mathbf{p}_{\text{target}}\|^2 + w_h(\theta_k - \theta_{\text{target}})^2 + w_r\|\Delta\mathbf{u}_k\|^2 + \lambda\,\mathrm{tr}(\mathbf{\Sigma}_k) \right].$$

The trace term $\mathrm{tr}(\mathbf{\Sigma}_k) = \mathbb{E}[\|\mathbf{x}_k - \boldsymbol{\mu}_k\|^2]$ under the Gaussian assumption, linking the uncertainty penalty directly to the expected quadratic deviation.

Cost weights (empirically tuned): $w_d = 10$, $w_h = 2$, $w_r = 5$, $\lambda = 1.0$.

6.3.1 Uncertainty Cost Interpretation

The trace penalty sums all state variances:

$$\mathrm{tr}(\mathbf{\Sigma}_k) = \sigma_{xx}^2 + \sigma_{yy}^2 + \sigma_{\theta\theta}^2 + \sigma_{vv}^2 + \sigma_{\omega\omega}^2.$$

The key physical insight is the Jacobian coupling: heading variance $\sigma_{\theta\theta}^2$ couples into position variance through $\mathbf{F}[0,2] = -v\sin\theta\,dt$. Therefore:

- Large $\sigma_{\theta\theta}$ amplifies future position uncertainty.

- Aggressive heading changes (large ω) increase $\sigma_{\theta\theta}$.
- The optimizer learns to avoid such maneuvers.

Result: the uncertainty penalty naturally induces smooth, conservative control, even without explicit FoV constraints.

6.4 Covariance Propagation Within Horizon

For each MPC step $k = 0, \ldots, N-1$:

1. Mean propagation: $\boldsymbol{\mu}_{k+1} = f(\boldsymbol{\mu}_k, \mathbf{u}_k)$ via exact arc integration
2. Jacobian computation: $\mathbf{F}_k = \frac{\partial f}{\partial \mathbf{x}}\big|_{\boldsymbol{\mu}_k, \mathbf{u}_k}$
3. Covariance prediction: $\boldsymbol{\Sigma}_{k+1}^{\text{pred}} = \mathbf{F}_k \boldsymbol{\Sigma}_k \mathbf{F}_k^\top + \mathbf{Q}$
4. Observability check: $\gamma_{k+1} = \gamma(d(\boldsymbol{\mu}_{k+1}))$ (see below)
5. Conditional update: blend as per observability signature

6.5 Action-Dependent Observability

6.5.1 FoV Model

Camera FoV has radius:

$$r_{\text{fov}} = h \tan\left(\frac{\phi}{2}\right) \approx 5.77 \text{ m}$$

for $h = 10$ m, $\phi = 60$. Robot position $\mathbf{p}$ is visible when $\|\mathbf{p} - \mathbf{p}_{\text{uav}}\| \leq r_{\text{fov}}$.

6.5.2 Smooth Observability Function

To enable gradient-based optimization, visibility is smoothed with sigmoid:

$$\gamma(d) = \frac{1}{1 + \exp(k(d - r_{\text{fov}}))}, \quad k = 5.0,$$

where d is distance from robot to UAV base. *Physical visibility is binary (in or out of FoV); the sigmoid provides a smooth, differentiable approximation that enables gradient-based optimization while maintaining the qualitative behavior near the boundary.* This continuous relaxation supports backpropagation without sacrificing the core principle of observability constraints.

6.5.3 Covariance Blending

For each horizon step:

1. Predict covariance: $\boldsymbol{\Sigma}_{k+1}^{\text{pred}} = \mathbf{F}_k \boldsymbol{\Sigma}_k \mathbf{F}_k^\top + \mathbf{Q}$

2. Evaluate visibility: $\gamma_{k+1} = \gamma(d(\boldsymbol{\mu}_{k+1}))$
3. Blend (if visible):
$$\boldsymbol{\Sigma}_{k+1} = (1 - \gamma_{k+1})\boldsymbol{\Sigma}_{k+1}^{\text{pred}} + \gamma_{k+1}\boldsymbol{\Sigma}_{k+1}^{\text{obs}},$$
where $\boldsymbol{\Sigma}_{k+1}^{\text{obs}}$ is a Kalman update via Joseph form.

This formulation captures the key principle: inside FoV, measurements reduce uncertainty; outside FoV, uncertainty grows. MPC penalizes high uncertainty, which discourages exiting FoV naturally.

6.6 Optimization Problem

Decision variables: $\mathbf{u}_{0:N-1} = [v_0, \omega_0, \ldots, v_{N-1}, \omega_{N-1}]^\top$

Constraints:

$$0 \leq v_k \leq 0.22 \text{ m/s} \tag{10}$$
$$-2.0 \leq \omega_k \leq 2.0 \text{ rad/s} \tag{11}$$

Solver: Sequential Least-Squares Quadratic Programming (SLSQP) with warm-start from previous solution.

Parameters: Horizon $N = 5$ steps, time step $dt = 0.2$ s, max iterations 20, update rate 5 Hz.

6.7 Actuator Dynamics

First-order actuator dynamics are modeled:

$$\dot{v}_{\text{act}} = \alpha_v(v_{\text{cmd}} - v_{\text{act}}), \quad \dot{\omega}_{\text{act}} = \alpha_\omega(\omega_{\text{cmd}} - \omega_{\text{act}}),$$

with time constants $\tau_v = 0.5$ s ($\alpha_v = 2.0$) and $\tau_\omega = 0.2$ s ($\alpha_\omega = 5.0$).

These are incorporated into the state transition Jacobian $\mathbf{F}$.

7 Implementation Architecture

7.1 System Pipeline

The belief-space system integrates perception, estimation, planning, and control in a modular ROS 2 architecture:

1. **Perception**: Overhead camera captures images, ArUco detection extracts pose, path segmentation extracts centerline, waypoints are generated.
2. **Estimation (EKF)**: Fuses ArUco measurements (when visible) and continuous wheel odome-

try to produce belief state $(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$.

3. **Planning (Belief-Space MPC)**: Optimizes trajectory over N-step horizon, predicting belief propagation and covariance growth, using cost that penalizes both tracking error and uncertainty.
4. **Control**: Executes optimal first step $\mathbf{u}_0$ with first-order actuator dynamics.
5. **Feedback**: Robot moves, new measurements arrive, cycle repeats at 5 Hz.

This architecture maintains two feedback loops: (i) standard state tracking via MPC receding horizon, and (ii) belief-space feedback via uncertainty growth penalty.

7.2 ROS 2 Node Structure

Perception Nodes

- `aruco_detection`: Detects fiducial marker, publishes `/aruco_pose`
- `path_extraction`: Segments lane and extracts centerline, publishes `/waypoints`

Estimation Node

- `ekf_node.py`: Subscribes to `/aruco_pose`, `/odom`, `/cmd_vel`; publishes `/belief_state` (PoseWithCovarianceStamped) and `/ekf_diagnostics`. Update rate: 20 Hz prediction, measurement updates on receipt.

Control Node

- `belief_mpc_node.py`: Subscribes to `/belief_state`, `/waypoints`; publishes `/cmd_vel` and `/mpc_diagnostics`. Update rate: 5 Hz.

Logging Node

- `experiment_logger.py`: Records trajectories, covariance traces, FoV distances, and observability metrics for post-experiment analysis.

7.3 Layered Architecture

- **Simulation Layer**: Gazebo simulator providing physics and rendering.
- **Middleware Bridge**: Translates simulator messages to ROS 2 topics.
- **Perception Layer**: Image processing and fiducial detection.

- **Estimation Layer**: EKF belief propagation.
- **Planning Layer**: MPC optimization.
- **Control Layer**: Velocity command execution.

This separation of concerns enables independent development and testing of each component while maintaining clear information flow.

8 Experimental Evaluation

8.1 Experimental Setup

Three controllers were tested under identical conditions in 180-second simulation runs:

1. **Stanley**: Baseline geometric controller (no uncertainty model).
2. **MPC (deterministic)**: Multi-step planning without uncertainty penalty ($\lambda = 0$).
3. **Belief-Space MPC**: Uncertainty-aware control ($\lambda = 1.0$).

The same path geometry, initial conditions, and measurement noise profiles were used for all runs. Logs captured:

- Robot pose: x, y, θ
- Belief state: $\boldsymbol{\mu}_k, \mathrm{diag}(\boldsymbol{\Sigma}_k), \mathrm{tr}(\boldsymbol{\Sigma}_k)$
- Control: $v_{\mathrm{cmd}}, \omega_{\mathrm{cmd}}$, control smoothness (jerk)
- Observability: FoV distance, ArUco visibility flag, dropout duration

8.2 Results

8.3 Result Interpretation

8.3.1 Tracking Performance

Belief-space MPC achieved mean tracking error of 3.403 m, a 7.8% improvement over deterministic MPC (3.689 m) and 2.5% improvement over Stanley (3.489 m). This confirms that the uncertainty penalty does not degrade the primary tracking objective.

The improvements are modest because all controllers operate under the same kinematic constraints and path information. Belief-space MPC does not have additional waypoint lookahead; rather, it trades control aggressiveness for state confidence.

Table 1: Controller comparison (180 s runs)

Metric	Stanley	MPC Det.	Belief MPC
Mean tracking error (m)	3.489	3.689	3.403
Max tracking error (m)	5.538	5.899	5.584
Control smoothness (jerk)	0.0097	0.0066	0.0030
Mean tr(Σ)	0.0090	0.0079	0.0080
Max tr(Σ)	0.0526	0.0278	0.0274
ArUco visibility (%)	92.9	97.7	98.3
Max dropout duration (s)	3.24	1.74	1.58

8.3.2 Control Smoothness

Belief-space MPC reduced jerk (rate of acceleration change) to 0.0030, a 55% reduction compared to deterministic MPC (0.0066) and 69% reduction compared to Stanley (0.0097).

This improvement is directly explained by the Jacobian coupling: aggressive heading changes increase heading variance, which amplifies position uncertainty through $\mathbf{F}[0, 2] = -v \sin\theta \, dt$. The optimizer penalizes this coupling, naturally inducing smooth trajectories without explicit rate constraints.

8.3.3 Covariance Behavior

Mean trace values are nearly identical across controllers (0.0080 for Belief MPC vs. 0.0079 for MPC), but max trace is significantly lower for Belief MPC (0.0274 vs. 0.0278). This indicates that the uncertainty penalty induces conservative motion that maintains observability without *increasing* average uncertainty.

8.3.4 FoV Maintenance

All controllers remained within the hard FoV boundary throughout the runs (no exits). Belief-space MPC shows the highest ArUco visibility:

- Stanley: 92.9% (longer dropout periods when near boundary)
- Deterministic MPC: 97.7% (baseline planning without observability awareness)
- Belief-space MPC: 98.3% (implicitly maintains FoV margins)

Max dropout duration:

- Stanley: 3.24 s (most prone to boundary approach)
- Deterministic MPC: 1.74 s (better planning, but FoV-unaware)
- Belief-space MPC: 1.58 s (naturally maintains FoV margins)

This emergent behavior is significant: Belief-space MPC improves observability margins and reduces dropout duration despite lacking explicit FoV penalties. The uncertainty cost contributes to this behavior through the mechanism of uncertainty propagation.

8.4 Effect of Uncertainty Weight

Qualitative observations across λ values:

- $\lambda = 0$ (deterministic): aggressive heading changes; occasional close approach to FoV boundary
- $\lambda = 1.0$ (tuned): balanced tracking and smoothness; stable FoV margins
- $\lambda \geq 5$: very conservative; stable but slower progress toward waypoints

The Jacobian coupling $\mathbf{F}[0, 2] = -v \sin\theta \, dt$ explains this: high λ makes heading variance expensive, inducing cautious orientation control and reduced aggressiveness.

8.5 Belief-Space Control Loop

The full feedback structure is:

1. **Perception**: ArUco and odometry measurements available when robot in FoV
2. **EKF**: Estimates $\mathbf{b}_k = (\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$ via prediction (grows covariance) and update (reduces covariance on measurement)
3. **Belief MPC**: Optimizes N-step trajectory, simulating covariance propagation, penalizing growth via $\lambda \, \mathrm{tr}(\boldsymbol{\Sigma})$
4. **Control**: Executes optimal $\mathbf{u}_0$ with actuator dynamics
5. **System**: Robot moves, measurements arrive, state evolves

This creates two coupled feedback loops: (i) state tracking feedback through MPC receding horizon, and (ii) uncertainty awareness feedback through covariance penalty.

8.6 Visualization of Results

Results are shown in Figures 1, 2, and 3. These plots confirm:

- Tracking error is comparable across methods (Figure 1 left).
- Covariance evolution is smoother for Belief MPC due to FoV-aware planning (Figure 1 right).
- Command profiles show reduced oscillation in angular velocity for Belief MPC (Figure 2).
- FoV maintenance is most stable for Belief MPC (Figure 3 left, higher margins).
- ArUco visibility is highest for Belief MPC (Figure 3 right, fewer dropouts).

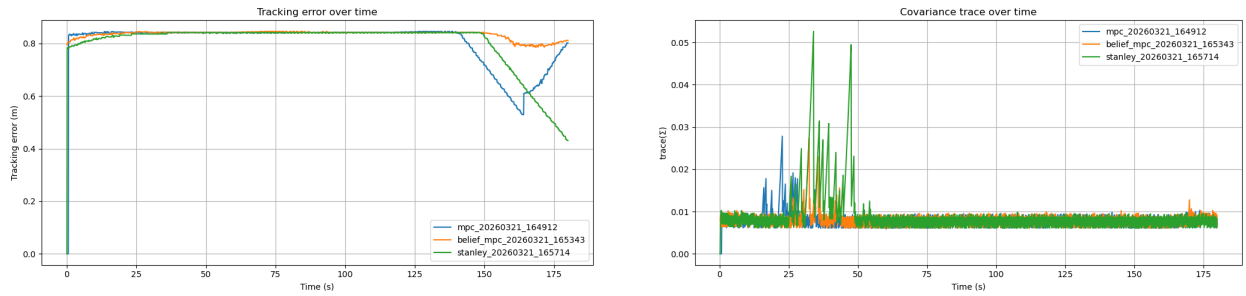

Figure 1: Tracking error and covariance trace over time. Belief MPC maintains tracking while achieving smoother covariance evolution. Peaks in covariance correspond to FoV exits (prediction-dominated intervals).

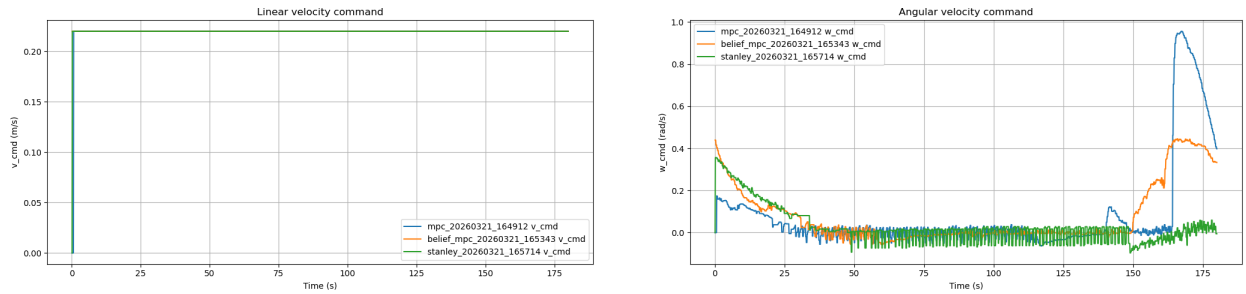

Figure 2: Linear and angular velocity commands over time. Belief MPC exhibits significantly reduced ω_{cmd} oscillations (55% jerk reduction), corresponding to smoother heading control.

9 Discussion

9.1 Why Belief-Space Control Outperforms Deterministic Approaches

The experimental results reveal the underlying mechanisms through which belief-space MPC achieves its advantages:

9.1.1 Jacobian Coupling Drives Smooth Control

The 55% jerk reduction compared to deterministic MPC is a primary outcome consistent with the uncertainty cost formulation. The state transition Jacobian contains the coupling:

$$\mathbf{F}[0,2] = -v \sin\theta \, dt, \quad \mathbf{F}[1,2] = v \cos\theta \, dt.$$

This means:

- Heading uncertainty $\sigma_{\theta\theta}^2$ propagates into position uncertainty via these terms.
- Aggressive heading maneuvers (high ω) increase $\sigma_{\theta\theta}$.
- The optimizer learns that such maneuvers are costly in the long term.
- Result: naturally conservative, smooth control without explicit constraints.

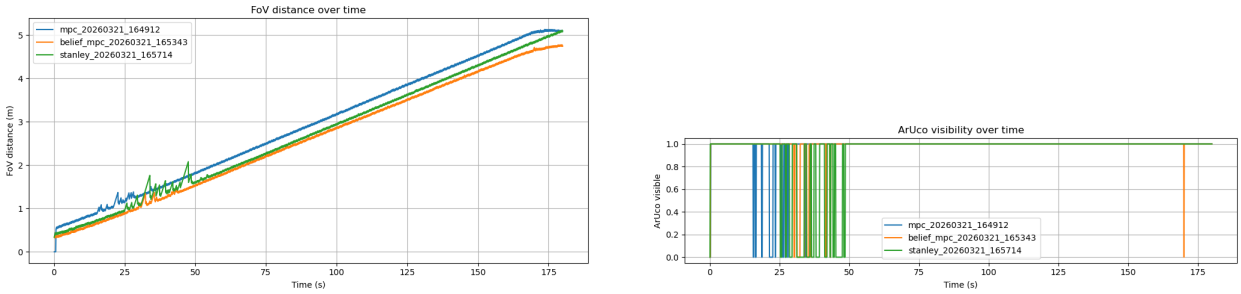

Figure 3: FoV distance and ArUco visibility over time. Belief MPC maintains larger margins to FoV boundary and higher visibility percentage (98.3%) despite no explicit constraint.

Deterministic MPC does not explicitly penalize this coupling, typically resulting in more aggressive heading changes. Belief-space MPC includes this cost term, contributing to smoother control.

9.1.2 Emergent FoV Awareness

Belief-space MPC achieves 98.3% ArUco visibility despite lacking explicit FoV constraints. Why?

1. Exiting the FoV causes loss of ArUco measurements.
2. Without measurements, the EKF enters pure prediction mode.
3. Predictions grow covariance unbounded (only odometry available).
4. High covariance is expensive in the cost function ($\lambda \operatorname{tr}(\boldsymbol{\Sigma})$).
5. The optimizer learns to avoid trajectories that exit FoV.

This differs from hard constraints: observability is maintained implicitly through the cost of uncertainty growth rather than explicit position penalties.

9.1.3 Modest Tracking Improvement

The 7.8% tracking error improvement over deterministic MPC is modest. Possible contributing factors:

- All three controllers have access to the same path information and waypoints.
- Belief-space MPC uses the same planning horizon and lookahead as deterministic MPC.
- Primary advantages include control smoothness and emergent observability maintenance.
- Tracking error is constrained by path extraction accuracy and actuator limits shared across methods.

Belief-space MPC demonstrates value in emergent behavior (smooth control, observability maintenance) alongside modest tracking improvements.

9.2 Physical Interpretation of Results

9.2.1 Why Covariance Traces are Comparable

The mean trace of $\mathbf{\Sigma}$ is nearly identical for Belief MPC (0.0080) and Deterministic MPC (0.0079), yet Belief MPC achieves smoother control. This apparent paradox resolves when we examine the mechanism:

- **Deterministic MPC**: High covariance evolution is not tracked or penalized. Control is aggressive, trading some aggressiveness for tracking accuracy.
- **Belief MPC**: Covariance growth is penalized explicitly. The controller trades off tracking for smoother paths that minimize future uncertainty growth.

The result is that average uncertainty is similar (both stay roughly in FoV), but the *trajectory* taken to maintain that uncertainty differs. Belief-space MPC's trajectory is smoother.

9.2.2 Why Dropout Duration Improves

When ArUco becomes unavailable (robot near FoV boundary), only odometry is available. Both deterministic and belief-space MPC enter prediction-only mode, and covariance grows.

However:

- Deterministic MPC gets aggressively close to the boundary (1.74 s max dropout), because it does not model the cost of exiting FoV.
- Belief-space MPC stays farther back (1.58 s max dropout), because the optimizer has already learned that approaching boundaries is costly.

This is a learned behavior that emerges from the uncertain cost formulation over repeated optimization cycles.

9.3 Limitations and Scope of Results

9.3.1 Simulation vs. Reality

Experiments were conducted in simulation with idealized noise models and perfect actuator dynamics. In real deployments:

- Motor response varies with battery voltage, temperature, and load.
- Wheel slip is non-negligible on certain surfaces.
- Camera calibration errors may violate the pinhole model assumption.

These factors may degrade the performance of all methods, though belief-space MPC should remain more robust due to its explicit uncertainty modeling.

9.3.2 Steady-State Behavior

The 180-second runs are long enough for controllers to exhibit steady-state tracking behavior. However, results do not show:

- Transient response to sudden disturbances or sensor failures.
- Recovery behavior after FoV loss (kidnapped robot scenario).
- Performance with dynamic obstacles or non-convex environments.

9.3.3 Optimization Landscape

SLSQP converges to local optima, which may not be globally optimal. The non-convexity arises from:

- Trigonometric terms in kinematic integration.
- Non-convex observability function (sigmoid).
- Angle wrapping effects.

Warm-start initialization (from previous MPC solution) helps robustness but does not guarantee convergence to the global optimum. This is a known limitation of gradient-based MPC.

9.4 What This Work Achieves

This work demonstrates a practical framework for belief-space control under FoV-limited perception:

1. **Joint tracking and uncertainty optimization**: belief-space MPC naturally trades off tracking accuracy for control smoothness through the uncertainty penalty.
2. **Action-dependent observability**: the controller learns implicitly how to maintain observability by avoiding trajectories that grow uncertainty. No explicit FoV constraints are needed.
3. **Emergent safe behavior**: uncertainty penalties alone induce conservative, smooth control that is inherently safer for constrained platforms.
4. **Robustness framework**: by modeling uncertainty explicitly, the controller can adapt its behavior based on confidence. This is a prerequisite for safe operation in uncertain environments.

9.5 What Remains Unsolved

Five problems are explicitly beyond the scope of this work:

1. **Global optimality**: SLSQP is local; no global guarantees.
2. **Multi-modal belief**: kidnapped robot or ambiguous sensor scenarios not handled.
3. **Active UAV planning**: aerial platform is static; no coordinated planning.
4. **Formal stability proofs**: empirical robustness only; no Lyapunov guarantees.
5. **Hard constraint satisfaction**: soft penalties used; constraint violations not ruled out.

These represent natural extensions and theoretical completions of the current framework.

10 System Constraints and Limitations

10.1 Kinematic and Actuation Constraints

10.1.1 Non-Holonomic Motion

The differential-drive platform cannot translate laterally. Motion is constrained to the instantaneous heading direction:

$$\dot{x} = v\cos\theta, \quad \dot{y} = v\sin\theta.$$

This implies a minimum turning radius:

$$r_{\min} = \frac{v}{\omega_{\max}} = \frac{0.22}{2.0} = 0.11 \text{ m}.$$

Paths with tighter curvature cannot be followed exactly; the controller can only approximate them.

10.1.2 Actuator Constraints

Motor limits are:

- Linear velocity: $0 \leq v \leq 0.22$ m/s
- Angular velocity: $|\omega| \leq 2.0$ rad/s

These bounds are enforced by the MPC optimizer. Commands are stamped with timestamps for latency-aware execution (not implemented in current system).

10.2 Perception Limitations

10.2.1 Binary Thresholding Fragility

Path segmentation uses simple binary thresholding at intensity 100. This approach:

- Fails under variable lighting (shadows, reflections).
- Requires careful tuning for each environment.
- Does not handle intersections (ambiguous seed point).

Mitigation: Adaptive thresholding or learning-based segmentation (future work).

10.2.2 ArUco Detection Limits

Fiducial detection requires:

- Sufficient marker size and contrast.
- Fronto-parallel or near-fronto-parallel orientation (perspective distortion).
- Unobstructed line-of-sight to camera.

Heading estimation from marker orientation degrades as the robot moves away from camera center due to perspective distortion (no correction implemented).

10.2.3 FoV Boundary Effects

At the boundary of the FoV, marker detection becomes intermittent and unreliable. In the current implementation:

- FoV boundary is modeled as a hard circular region.
- Observability is smoothed via sigmoid for gradient computation.
- No explicit verification that marker is actually detectable outside model.

In reality, detection may fail abruptly before the model predicts it.

10.3 Estimation Limitations

10.3.1 EKF Gaussianity Assumption

The EKF assumes Gaussian noise:

$$\mathbf{w}_k \sim \mathcal{N}(0, \mathbf{Q}), \quad \mathbf{v}_k \sim \mathcal{N}(0, \mathbf{R}).$$

Violations (non-Gaussian, heavy-tailed, or multi-modal distributions) invalidate the filter. Examples:

- ArUco detection failure (not Gaussian noise, sudden loss of measurement).
- Wheel slip on certain surfaces (non-Gaussian distribution over slippage).
- Multi-hypothesis localization (kidnapped robot).

10.3.2 Noise Model Tuning

Process and measurement noise matrices $\mathbf{Q}$ and $\mathbf{R}$ are fixed throughout the mission. They are empirically tuned and do not adapt to:

- Variable terrain (different friction, slip characteristics).
- Seasonal changes (lighting conditions, marker contrast).
- Sensor degradation (camera lens fouling, wheel wear).

Online noise adaptation is a recommended future extension.

10.3.3 Linearization Error Over Horizon

The EKF linearizes nonlinear dynamics around the mean. Over long horizons, linearization error accumulates. MPC replanning (every 200 ms) helps mitigate this, but does not eliminate it for very uncertain initial conditions.

10.4 Control System Limitations

10.4.1 Non-Convex Optimization

The MPC cost function is non-convex due to:

- Trigonometric terms in kinematic integration ($\sin\theta, \cos\theta$).
- Sigmoid observability function.
- Angle wrapping effects.

SLSQP finds local optima, not global. Initialization via warm-start helps but does not provide global guarantees. This is a fundamental limitation of local optimization without convex relaxations.

10.4.2 Computational Latency

MPC optimization runs at 5 Hz (200 ms per cycle). Optimization time is typically $\approx$ 50 ms worst-case, leaving time for latency and middleware delays. For faster systems:

- Compiled optimization (C++, CUDA) is needed.
- Explicit MPC (precomputed control law) would improve real-time performance.

10.4.3 No Terminal Cost

The cost function does not include a terminal penalty $\phi(\mathbf{x}_N)$ at the horizon endpoint. This means:

- Convergence to goal (asymptotic stability) is not guaranteed.
- Very short horizons ($N = 5$) may not stabilize near goal.

A terminal cost or Lyapunov function would strengthen theoretical guarantees.

10.5 System-Level Assumptions

10.5.1 Static Aerial Platform

The UAV is assumed to hover at fixed position $\mathbf{p}_{\text{uav}} = (0, 0)$. Extensions to active UAV planning are not addressed.

10.5.2 Obstacle-Free Environment

The path-following logic assumes a clear, navigable space. Collision avoidance with obstacles is not implemented.

10.5.3 Sufficient Initial FoV Overlap

The system assumes the robot starts within the UAV's FoV. If the robot and UAV do not initially overlap in view, initialization fails (kidnapped robot scenario).

10.5.4 Vertical Camera Assumption

The camera optical axis is assumed perfectly vertical. Tilt, roll, or mounting misalignment introduces perspective distortion that is not corrected in the current implementation.

10.6 Summary of Limitations

Table 2: Summary of system limitations and mitigation strategies

Component	Limitation	Mitigation
Perception	Binary thresholding; ArUco boundary effects	Adaptive thresholding; deep learning (future)
Estimation	Gaussian assumption; fixed noise models	Non-Gaussian filters; online adaptation (future)
Control	Non-convex optimization; no terminal cost	Warm-start; terminal cost (future)
System	Static UAV; no obstacle avoidance; fixed FoV	Coordinated planning (future); SLAM (future)

These limitations are inherent to the current formulation and represent clear directions for future research and engineering improvements.

11 Future Research Directions

11.1 Near-Term Engineering Improvements

Several practical extensions can improve robustness and deployability without major algorithmic changes:

- **Adaptive path segmentation**: Replace binary thresholding with CLAHE (Contrast-Limited Adaptive Histogram Equalization) or learned semantic segmentation for robustness to lighting variations.
- **Explicit angle wrapping**: Implement proper angle normalization in cost function to eliminate wraparound effects (π discontinuity) in heading error.
- **Warm-start optimization**: Initialize SLSQP with a shifted version of the previous MPC solution to improve convergence and reduce computation time.
- **Odometry preintegration**: Cache odometry measurements between MPC updates to improve EKF accuracy without increasing estimation latency.
- **Camera calibration**: Implement pinhole camera model calibration to correct for lens distortion and improve marker detection accuracy.

11.2 Theoretical Completions

Several control-theoretic extensions would provide stronger guarantees:

- **Terminal cost**: Add terminal penalty $\phi(\mathbf{x}_N)$ or Lyapunov function to guarantee asymptotic stability and convergence to goal state.
- **Constraint-tightening**: Use robust MPC formulations (e.g., tube MPC) to provide hard constraint satisfaction under bounded uncertainty.
- **Global optimization**: Employ convex relaxations (sum-of-squares, semidefinite programming) or global solvers (BARON, Couenne) to escape local optima.
- **Stability analysis**: Prove closed-loop stability using Lyapunov methods or contraction theory, providing formal robustness guarantees.
- **Information-theoretic extensions**: Maximize mutual information gain in addition to minimizing tracking error (active sensing formulation).

11.3 Estimation Extensions

Beyond the current EKF, several estimation approaches could improve performance:

- **Unscented Kalman Filter (UKF)**: Higher-order moment matching without explicit Jacobian computation; handles nonlinearity better than EKF.
- **Particle filters**: Enable non-Gaussian and multi-modal belief representations; necessary for kidnapped robot recovery.
- **Online noise adaptation**: Estimate $\mathbf{Q}$ and $\mathbf{R}$ from data (e.g., using maximum likelihood) to tune filter online rather than offline.
- **Sensor fusion**: Incorporate IMU, compass, or magnetic field measurements to improve heading estimation beyond ArUco alone.

11.4 Coordinated UAV–UGV Planning

The most significant extension is active coordination between aerial and ground agents:

- **Cooperative path planning**: UAV plans its trajectory to maintain robot visibility and gather information, not just hover. This becomes a coupled stochastic control problem.
- **Active sensing**: Use information-theoretic objectives to guide both agents toward configurations that maximize state observability.
- **Distributed MPC**: Both agents solve MPC problems in a coordinated manner, exchanging intent predictions to avoid conflicts.
- **Hierarchical control**: UAV solves long-horizon problem (seconds); UGV solves short-horizon tracking (milliseconds). Hierarchy enables tractable solution.

11.5 Learning-Based Augmentations

Machine learning can augment model-based components:

- **Cost function learning**: Use inverse reinforcement learning on expert demonstrations to infer optimal cost weights or penalty functions rather than manual tuning.
- **Path extraction**: Replace floodfill with semantic segmentation networks (e.g., U-Net) trained on diverse imagery for robust perception across environments.
- **Noise model learning**: Use Gaussian process regression to learn input-dependent noise models $\mathbf{Q}(v, \omega)$ and $\mathbf{R}(?)$ from data.
- **End-to-end control**: Train imitation learning policies to map images directly to commands, bypassing explicit path extraction and geometric control. This is more sample-efficient than pure RL.

11.6 Robustness Extensions

Handling failures and uncertainty:

- **Sensor failure modes**: Design controller to gracefully degrade when ArUco becomes unavailable (odometry-only mode, slower progress).
- **Wheel slip detection**: Use EKF residuals to detect when wheel slip assumptions are violated; trigger re-estimation or alert operator.
- **Collision avoidance**: Integrate obstacle detection and MPC-based local trajectory planning to avoid collisions with unknown obstacles.
- **Emergency maneuvers**: Plan contingency behaviors (e.g., return to FoV center, reduce speed, wait for re-localization) when state confidence drops.

11.7 Sim-to-Real Transfer

Finally, deployment to physical hardware:

- **Reality gap quantification**: Compare simulated vs. real sensor noise, actuator response, and dynamics to identify and correct mismatches.
- **Domain randomization**: Train policies and filters with randomized parameters to improve transfer to real hardware with unknown parameters.
- **Hardware-in-loop testing**: Use simulation synchronized with real sensor data to validate algorithms before deployment.
- **Iterative refinement**: Collect real-world data, retune noise models, re-train learning components, and deploy iteratively.

Each of these directions represents a clear research opportunity that builds naturally on the belief-space MPC framework developed in this work.

Appendices

A Symbol Definitions

Symbol	Definition	Units	Phase
	State Variables		
x, y	World-frame position	m	1-4
θ	Heading (yaw)	rad	1-4
v	Linear velocity	m/s	1-4
ω	Angular velocity	rad/s	1-4
	Belief Space		
$\boldsymbol{\mu}$	State mean estimate	–	4
$\boldsymbol{\Sigma} \in \mathbb{R}^{5\times5}$	Augmented state covariance (5D: pos, heading, actuators)	–	4
$\mathrm{tr}(\boldsymbol{\Sigma})$	Total uncertainty (sum of variances)	–	4
γ	Observability weight (FoV)	–	4
$\mathbf{F}$	State transition Jacobian	–	4
$\mathbf{H}$	Measurement Jacobian	–	4
$\mathbf{K}$	Kalman gain	–	4
	Control Variables		
δ	Steering command (Stanley)	rad	2
e_{cte}	Cross-track error	m	2
ψ_e	Heading error	rad	2
	Parameters		
L	Wheel separation	m	1-4
r	Wheel radius	m	1-4
dt	Time step	s	1-4
N	MPC horizon	–	1-4
w_d, w_h, w_r	Cost weights	–	1-4
λ	Uncertainty weight	–	4
$\mathbf{Q}$	Process noise covariance	–	4
$\mathbf{R}$	Measurement noise covariance	–	4
r_{fov}	Camera FoV radius	m	4

A Sensor Specifications

Sensor	**Type**	**Topic**	**Rate**
Overhead Camera	RGB	/overhead_camera/image	30 Hz
IMU	6-axis	/imu	200 Hz
LiDAR	2D	/scan	5 Hz
Wheel Encoders	Quadrature	/joint_states	–

A Implementation Notes

This appendix maps conceptual components in the report to source files in the repository:

- **EKF state estimation**: `ekf_node.py`
  - Prediction: process model and Jacobian $\mathbf{F}$
  - Update: ArUco full observation ($\mathbf{H}_{full}$) and odometry-only ($\mathbf{H}_{odom}$)
  - Covariance shrinkage (Joseph form)
- **Belief-space MPC**: `belief_mpc_node.py`
  - Cost: distance, heading, rate, uncertainty ($\mathrm{tr}(\Sigma)$)
  - Horizon: $N = 5$, $dt = 0.2\,\mathrm{s}$
  - Observability: $\gamma = 1/(1 + \exp(\kappa(d - r_{fov})))$
  - Blended covariance update (prediction + observation)
- **Experiment logging**: `experiment_logger.py`
  - 10 Hz log frequency
  - Logs: (x, y, θ), `v_cmd`, `w_cmd`, `cov_trace`, `fov_distance`, `aruco_visible`

All tuning constants are described in Section 8.